LAPORAN PRAKTIKUM MOBILE PROGRAMMING

Laporan ini disusun untuk memenuhi tugas
Praktikum Mobile Programming Modul 11

Dosen Pengampu:

Khadijah Fahmi Hayle, S.Kom., M.Kom



Disusun oleh:

Muhammad Rizqullah Saputra
240605110102

PRODI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI
MAULANA MALIK IBRAHIM MALANG
2026

PENDAHULUAN

Modul 11 membahas manajemen state pada Flutter menggunakan GetX melalui pembuatan aplikasi tasbih digital. State yang dikelola berupa nilai penghitung dan kemajuan hitungan menuju batas maksimum. Materi ini menekankan hubungan antara controller, variabel reaktif, dan widget pengamat agar perubahan data dapat ditampilkan tanpa pemanggilan setState() pada halaman Home.

Aplikasi memiliki tampilan utama berupa angka penghitung, progress bar, tombol berikon fingerprint untuk menambah hitungan, dan tombol reset. Logika penambahan serta pengaturan ulang hitungan ditempatkan di dalam TasbihController, sedangkan halaman Home bertanggung jawab menyusun antarmuka. Pemisahan tersebut membuat alur perubahan data lebih mudah ditelusuri dan mengurangi pencampuran logika dengan susunan widget.

Pembahasan laporan mengikuti langkah praktikum mulai dari persiapan proyek dan dependensi, penyusunan tampilan, pembuatan controller, hingga penerapan Obx. Bagian tugas menjelaskan cara kerja state reaktif dan membandingkannya dengan setState(). Dokumentasi hasil menggunakan dua tangkapan layar yang disertakan, yaitu kondisi penghitung 5 untuk langkah kerja dan kondisi penghitung 13 untuk tugas.

TUJUAN PRAKTIKUM

1. Memahami konsep manajemen state dalam aplikasi Flutter menggunakan GetX.
2. Membedakan pembaruan state menggunakan setState() dengan pendekatan reaktif GetX.
3. Menerapkan controller serta variabel reaktif counter dan progress untuk mengelola data aplikasi.
4. Menggunakan widget Obx untuk menampilkan perubahan nilai yang dipantau secara otomatis.
5. Menerapkan konsep tersebut pada aplikasi tasbih digital dengan fungsi penambahan dan pengaturan ulang hitungan.

DASAR MANAJEMEN STATE DENGAN GETX

A. State dan Pembaruan Tampilan

State adalah data yang dapat berubah dan memengaruhi tampilan aplikasi. Pada tasbih digital, angka penghitung berubah saat pengguna menekan tombol, sedangkan progress bar menggambarkan perbandingan hitungan terhadap batas 33. Data dan tampilan perlu diperbarui secara konsisten agar angka dan indikator kemajuan tidak menunjukkan kondisi yang berbeda.

Pada pendekatan setState(), data umumnya disimpan di dalam kelas State dari StatefulWidget. Perubahan data dilakukan di dalam callback setState() agar Flutter menjadwalkan pembangunan ulang subtree widget yang bersangkutan. Pendekatan ini tetap sesuai untuk state lokal sederhana; pemilihannya bergantung pada kebutuhan aplikasi, bukan berarti setState() selalu kurang baik.

B. Controller dan Variabel Reaktif

GetX menyediakan kelas GetxController untuk menempatkan logika aplikasi di luar widget tampilan. Akhiran .obs membuat suatu nilai menjadi reaktif atau observable. Pada praktikum ini, counter dan progress dibuat dari nilai awal 0.0 sehingga keduanya merupakan state numerik reaktif. Nilai yang tersimpan dibaca atau diubah melalui properti .value.

C. Fungsi Widget Obx

Obx membungkus widget yang membaca variabel reaktif. Ketika nilai yang dibaca berubah, bagian tampilan di dalam Obx dapat dibangun ulang. Dalam aplikasi ini, Obx pertama mengamati counter untuk menampilkan angka, sedangkan Obx kedua mengamati progress untuk mengatur panjang indikator. Halaman Home tetap dapat ditulis sebagai StatelessWidget karena perubahan data dikelola oleh controller.

PERSIAPAN PROYEK DAN DEPENDENSI

A. Membuat Proyek dan Struktur Folder

Alat yang digunakan dalam materi meliputi Flutter SDK, Android Studio atau VS Code, serta emulator atau perangkat untuk menjalankan aplikasi. Dokumentasi yang dilampirkan pada laporan ini memperlihatkan aplikasi lokal melalui browser dengan bingkai perangkat. Karena itu, tampilan tersebut tidak dinyatakan sebagai bukti pemasangan aplikasi pada perangkat Android fisik.

Pada langkah awal, modul menyebut nama proyek tasbih_getx, sedangkan contoh struktur dan format laporannya memakai getx_app. Pada direktori praktikum, implementasi disimpan terpisah dalam proyek langkah_kerja dan tugas. Keduanya memiliki susunan kode utama yang sama di dalam folder lib.

1. Buat atau buka proyek Flutter untuk aplikasi tasbih digital.

2. Siapkan folder model, view, dan viewmodel di dalam lib.

3. Tempatkan home.dart di folder view dan tasbih_controller.dart di folder viewmodel. File main.dart tetap menjadi titik masuk aplikasi.

```
lib/
  main.dart
  model/
  view/
    home.dart
  viewmodel/
    tasbih_controller.dart
```

Folder model disiapkan sesuai struktur modul untuk kelas data. Contoh tasbih ini belum memerlukan kelas model terpisah karena state yang diolah hanya berupa penghitung dan progress. Folder view menyimpan tampilan, sedangkan viewmodel menyimpan controller beserta logika perubahan datanya.

B. Menambahkan Dependensi pada pubspec.yaml

Paket get menyediakan GetMaterialApp, GetxController, variabel reaktif, dan Obx. Modul juga mencantumkan simple_circular_progress_bar, tetapi indikator yang dipakai pada tampilan akhir adalah LinearProgressIndicator dari Flutter. Oleh sebab itu, paket tambahan tersebut bukan komponen yang menggambar progress bar pada contoh ini.

```
dependencies:
  flutter:
    sdk: flutter
  get: ^4.6.5
  simple_circular_progress_bar: ^1.0.2
```

1. Tambahkan dependensi di atas pada bagian dependencies dengan indentasi YAML yang benar.

2. Simpan pubspec.yaml dan jalankan flutter pub get dari direktori proyek.

3. Pada proyek tugas, dependensi get tetap digunakan, sedangkan simple_circular_progress_bar tidak diperlukan karena tampilan menggunakan LinearProgressIndicator.

```
flutter pub get
```

C. Menyiapkan Entry Point dengan GetMaterialApp

File main.dart memanggil runApp() untuk menjalankan MyApp. GetMaterialApp digunakan sebagai widget aplikasi dan Home dijadikan halaman awal. Konfigurasi title, tema hijau, dan penghilangan banner debug mengikuti contoh langkah kerja. Potongan berikut berasal dari main.dart pada proyek langkah_kerja.

```
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import 'view/home.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Tasbih Digital GetX',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.green),
        useMaterial3: true,
      ),
      home: Home(),
    );
  }
}
```

GetMaterialApp mendukung integrasi berbagai fasilitas GetX pada tingkat aplikasi. Namun, konsep pembaruan state pada contoh ini tetap terletak pada variabel .obs, controller, dan Obx; perubahan angka tidak terjadi hanya karena MaterialApp diganti menjadi GetMaterialApp.

LANGKAH KERJA APLIKASI TASBIH DIGITAL

A. Menyusun Tampilan Awal Home

Halaman Home dibuat sebagai StatelessWidget dengan latar hijau. Susunan utamanya menggunakan Scaffold, SafeArea, Center, dan Column. Pada tahap awal dalam modul, teks penghitung masih bernilai tetap 0, progress bar belum terhubung ke nilai state, dan callback tombol masih kosong. Tahap ini dipakai untuk menyiapkan susunan antarmuka sebelum logika reaktif ditambahkan.

1. Import material.dart, get.dart, dan tasbih_controller.dart pada home.dart.

2. Gunakan Column dengan mainAxisAlignment.center untuk menyusun angka, progress bar, serta tombol hitung.

3. Atur angka dengan ukuran font 250, progress bar berwarna amber, dan tombol hitung berikon fingerprint.

4. Tambahkan FloatingActionButton berikon refresh sebagai tombol reset.

B. Membuat TasbihController

TasbihController merupakan turunan GetxController. Counter menyimpan jumlah hitungan, progress menyimpan persentase kemajuan, dan maxCount menetapkan batas 33. Kode controller di bawah sama pada proyek langkah_kerja dan tugas.

```
import 'package:get/get.dart';

class TasbihController extends GetxController {
  var counter = 0.0.obs;
  var progress = 0.0.obs;
  final double maxCount = 33;

  void incrementCounter() {
    if (counter < maxCount) {
      counter.value++;
      progress.value = (counter.value / maxCount) * 100;
    }
  }

  void resetCounter() {
    counter.value = 0;
    progress.value = 0;
  }
}
```

Method incrementCounter() hanya menambah hitungan ketika nilainya masih lebih kecil daripada maxCount. Sesudah counter bertambah satu, progress dihitung kembali dengan rumus (counter / maxCount) × 100. Saat nilai sudah 33, kondisi tersebut tidak terpenuhi sehingga penambahan berikutnya tidak menaikkan angka melewati batas. Penjelasan ini merupakan perilaku yang ditentukan oleh logika kode.

Method resetCounter() menetapkan counter dan progress menjadi nol. Kedua nilai diperbarui bersama agar angka dan indikator kembali ke kondisi awal yang konsisten. Controller tidak memanggil setState() karena perubahan .value pada variabel reaktif dapat diamati langsung oleh Obx.

C. Menghubungkan Home dengan Controller

Instance TasbihController didaftarkan melalui Get.put() sehingga dapat digunakan oleh halaman Home. Pada source praktikum yang digunakan, baris pendaftaran ditempatkan di awal method build(). Setelah controller tersedia, callback tombol dan widget pengamat dapat merujuk pada objek tersebut.

```
final TasbihController controller =
    Get.put(TasbihController());
```

Get.put() pada bagian ini menyediakan objek controller melalui mekanisme dependency management GetX. Pendaftaran objek dan pembaruan tampilan memiliki peran berbeda: Get.put() menyediakan controller, sedangkan .obs bersama Obx menghubungkan perubahan state ke UI.

D. Menampilkan Counter Secara Reaktif

Widget Text dibungkus dengan Obx dan membaca controller.counter.value. Pemanggilan round() mengubah angka pecahan yang disimpan pada RxDouble menjadi tampilan bilangan bulat. Dengan demikian, hitungan terlihat sebagai 5 atau 13, bukan 5.0 atau 13.0.

```
Obx(
  () => Text(
    '${controller.counter.value.round()}',
    style: const TextStyle(fontSize: 250),
  ),
),
```

E. Menghubungkan Progress Bar dengan Nilai State

Nilai progress pada controller disimpan dalam skala 0 sampai 100. Sementara itu, properti value milik LinearProgressIndicator menggunakan rentang 0.0 sampai 1.0. Nilai progress karena itu dibagi 100 sebelum diteruskan ke widget. Obx memantau perubahan progress agar panjang indikator mengikuti perubahan hitungan.

```
Obx(
  () => Padding(
    padding: const EdgeInsets.symmetric(horizontal: 40.0),
    child: LinearProgressIndicator(
      value: controller.progress.value / 100,
      backgroundColor: Colors.white54,
      color: Colors.amberAccent.shade400,
      minHeight: 15,
      borderRadius: BorderRadius.circular(10),
    ),
  ),
),
```

Secara matematis, nilai indikator sama dengan counter / 33. Pada hitungan 5, persentase yang dihitung adalah sekitar 15,15% dengan value sekitar 0,1515. Pada hitungan 13, persentasenya sekitar 39,39% dengan value sekitar 0,3939. Persentase ini merupakan hasil perhitungan dari kode, bukan label persentase yang ditampilkan pada screenshot.

F. Menghubungkan Tombol Hitung dan Reset

Tombol fingerprint dibuat menggunakan InkWell agar ketukan pengguna dapat diarahkan ke incrementCounter(). Container putih serta ClipRRect membentuk area tombol dengan sudut membulat. Ikon fingerprint merupakan simbol tombol hitung, bukan implementasi autentikasi biometrik.

```
ClipRRect(
  borderRadius: const BorderRadius.all(Radius.circular(50)),
  child: InkWell(
    onTap: controller.incrementCounter,
    child: Container(
      decoration: const BoxDecoration(color: Colors.white),
      padding: const EdgeInsets.all(30),
      child: const Icon(Icons.fingerprint, size: 100),
    ),
  ),
),
```

Tombol reset memakai FloatingActionButton. Properti onPressed dihubungkan langsung ke resetCounter() sehingga controller bertanggung jawab mengubah kedua state ke nol, sementara Obx menangani pembaruan widget yang bergantung pada nilainya.

```
floatingActionButton: FloatingActionButton(
  onPressed: controller.resetCounter,
  backgroundColor: Colors.white,
  child: const Icon(
    Icons.refresh_outlined,
    color: Colors.black,
  ),
),
```

G. Menjalankan Program dan Mengamati Hasil

1. Pilih target yang tersedia pada Flutter, kemudian jalankan proyek dari IDE atau dengan flutter run.

2. Amati nilai awal counter dan indikator kemajuan.

3. Gunakan tombol fingerprint untuk menambah hitungan, lalu amati hubungan antara angka dan progress bar.

4. Gunakan tombol reset untuk memeriksa kondisi kembali ke nol dan lanjutkan pemeriksaan batas maksimum sesuai logika controller.

Urutan di atas merupakan prosedur pemeriksaan fungsi. Bukti visual yang disertakan pada laporan ini adalah kondisi counter 5 dan 13. Screenshot kondisi nol setelah reset serta kondisi batas 33 tidak disertakan, sehingga kedua skenario tersebut tidak dicatat sebagai hasil uji yang telah terdokumentasi.

TUGAS: PENERAPAN DAN ANALISIS GETX

A. Ruang Lingkup Tugas Modul 11

Tugas pada bagian 11.4 modul meminta laporan hasil praktikum yang menjelaskan tujuan, proses penerapan GetX, penggunaan controller, variabel reaktif, serta Obx. Laporan juga diminta membandingkan pendekatan setState() dan GetX, lalu menyertakan screenshot perubahan angka penghitung serta progress bar. Dengan demikian, fokus tugas adalah menjelaskan implementasi yang dikembangkan, bukan menambahkan fitur aplikasi di luar materi.

Pada proyek tugas, logika TasbihController tetap sama dengan langkah kerja. Perbedaan tampilan yang terlihat pada source adalah penambahan AppBar berjudul Tasbih Digital GetX, sedangkan title pada GetMaterialApp menjadi getx_app. Penambahan AppBar tidak mengubah mekanisme counter maupun perhitungan progress.

```
appBar: AppBar(
  title: const Text('Tasbih Digital GetX'),
  backgroundColor: Colors.green,
  foregroundColor: Colors.white,
),
```

B. Alur Controller, Variabel Reaktif, dan Obx

1. Ketukan pada tombol fingerprint memanggil incrementCounter() melalui callback onTap.

2. Controller memeriksa batas maxCount sebelum menambah counter.value.

3. Controller menghitung ulang progress.value berdasarkan nilai counter terbaru.

4. Obx yang membaca counter membangun ulang teks angka, sedangkan Obx yang membaca progress memperbarui indikator.

5. Ketika resetCounter() dipanggil, nilai counter dan progress diatur ke nol melalui controller yang sama.

Alur tersebut memisahkan tanggung jawab: view menerima interaksi dan menyajikan data, sedangkan controller menentukan aturan perubahan data. Variabel reaktif menjadi penghubung antara keduanya. Istilah pembaruan real time pada konteks ini merujuk pada respons tampilan terhadap perubahan state lokal, bukan sinkronisasi data dengan server atau jaringan.

C. Perbandingan dengan Pendekatan setState()

Jika counter disimpan secara lokal pada StatefulWidget, bentuk pembaruannya dapat ditulis menggunakan setState(). Contoh berikut hanya dipakai untuk menjelaskan perbandingan konsep; aplikasi hasil praktikum tetap menggunakan GetX.

```
double counter = 0;
double progress = 0;
final double maxCount = 33;

void incrementCounter() {
  setState(() {
    if (counter < maxCount) {
      counter++;
      progress = (counter / maxCount) * 100;
    }
  });
}
```

Pada contoh setState(), data dan method pembaruan berada di dalam kelas State. Pemanggilan setState() memberi tahu Flutter bahwa subtree widget terkait perlu dibangun ulang. Pada GetX, data ditempatkan dalam controller dan dijadikan reaktif dengan .obs; perubahan nilainya kemudian diamati oleh Obx tanpa callback setState() di halaman Home.

1. Lokasi state: setState() lazim digunakan untuk state lokal dalam kelas State; GetX menempatkan state dan logika pada controller terpisah.

2. Pemicu pembaruan: setState() menggunakan pemanggilan eksplisit, sedangkan GetX menggunakan perubahan variabel Rx yang dibaca Obx.

3. Bagian UI yang dibangun ulang: setState() menjadwalkan build pada State terkait; GetX memungkinkan pengamatan dibatasi pada bagian UI yang dibungkus Obx.

4. Dependensi: setState() tersedia langsung pada Flutter; GetX memerlukan paket get.

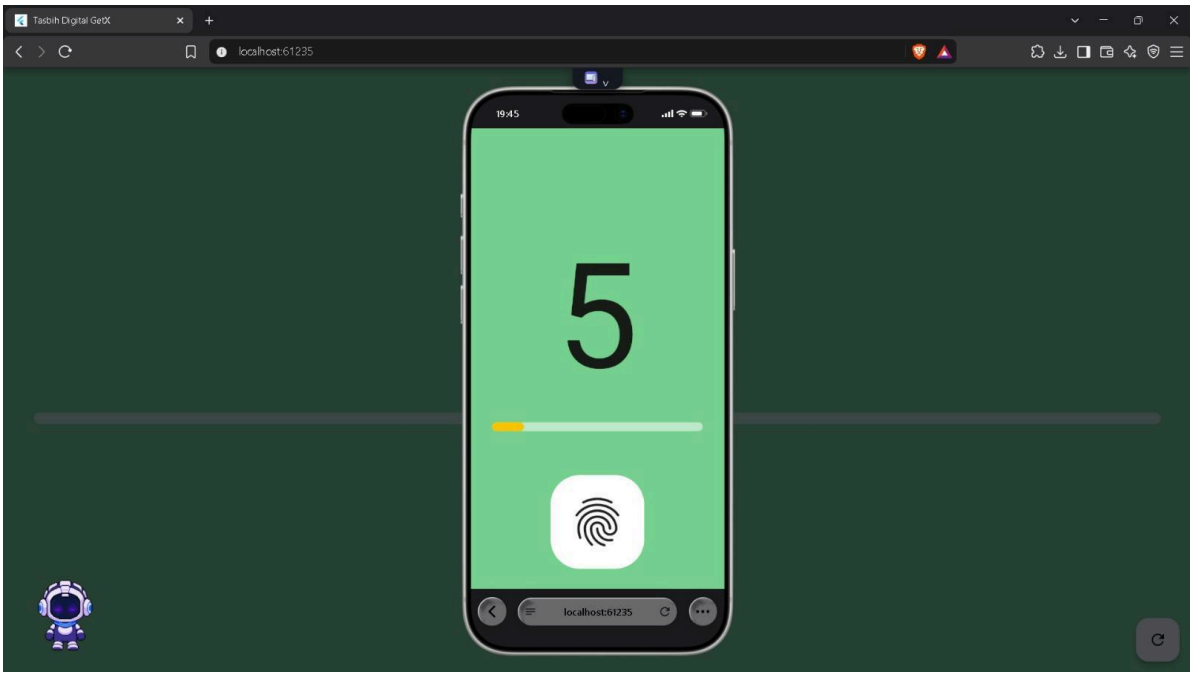
5. Pemilihan pendekatan: setState() cukup untuk kebutuhan lokal sederhana, sedangkan controller GetX dapat membantu pemisahan logika dan pengelolaan state yang dipakai di beberapa bagian aplikasi.

Perbandingan ini bersifat struktural dan konseptual. Tidak dilakukan pengukuran performa pada kedua pendekatan, sehingga laporan tidak menyatakan bahwa GetX pasti lebih cepat atau lebih hemat memori pada setiap aplikasi.

HASIL TAMPILAN APLIKASI

A. Hasil Langkah Kerja: Counter 5

Tangkapan layar berikut digunakan sebagai dokumentasi bagian langkah kerja. Tampilan menunjukkan angka 5 berukuran besar, progress bar dengan bagian terisi berwarna kuning, serta tombol berikon fingerprint. Aplikasi ditampilkan melalui browser lokal dalam bingkai perangkat, dengan latar antarmuka berwarna hijau.

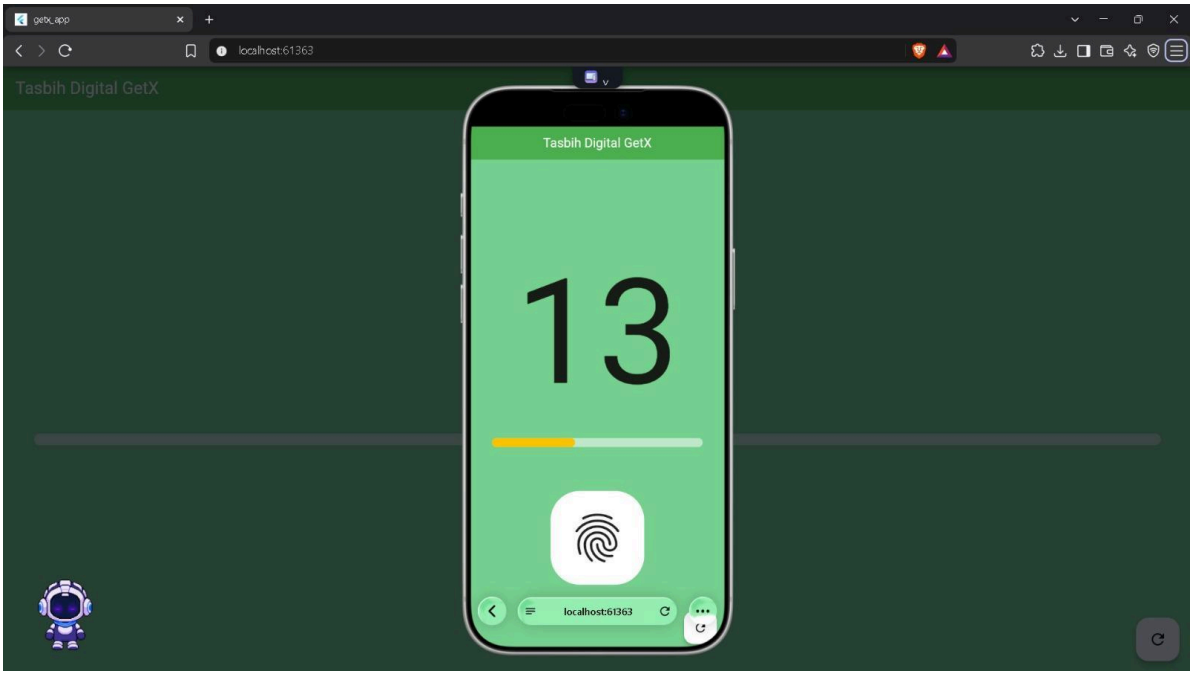


Gambar 1. Hasil langkah kerja aplikasi tasbih digital pada nilai penghitung 5.

Nilai 5 menunjukkan kondisi counter yang didokumentasikan pada langkah kerja. Berdasarkan rumus pada controller, kondisi tersebut setara dengan kemajuan 5/33 atau sekitar 15,15%. Angka persentase ini dihitung dari implementasi; screenshot menampilkan angka penghitung dan indikator, bukan teks persentase.

B. Hasil Tugas: Counter 13

Tangkapan layar kedua digunakan sebagai dokumentasi bagian tugas sesuai pembagian hasil praktikum. Tampilan menunjukkan nilai penghitung 13 dan progress bar di bawahnya. Bagian atas aplikasi menampilkan AppBar berjudul Tasbih Digital GetX, sesuai komponen tambahan yang terdapat pada source proyek tugas.



Gambar 2. Hasil tugas aplikasi tasbih digital pada nilai penghitung 13.

Berdasarkan rumus counter / 33, nilai 13 setara dengan kemajuan sekitar 39,39%. Kedua gambar mendokumentasikan dua nilai state yang berbeda pada proyek langkah kerja dan tugas. Gambar tersebut tidak diperlakukan sebagai rekaman berurutan dari satu sesi, serta tidak menjadi bukti tersendiri untuk pengujian reset, batas maksimum, atau pemasangan pada perangkat fisik.

KESIMPULAN

1. Manajemen state pada aplikasi tasbih digital disusun menggunakan GetxController sehingga logika penghitung dipisahkan dari widget tampilan.

2. Variabel counter dan progress dibuat reaktif dengan .obs. Pembaruan .value menyediakan perubahan data yang dapat diamati tanpa setState() pada Home.

3. Widget Obx menghubungkan counter dengan teks angka dan progress dengan LinearProgressIndicator. Konversi skala persentase menjadi rentang 0 sampai 1 diperlukan agar indikator sesuai dengan nilai hitungan.

4. Method incrementCounter() mengatur penambahan hingga batas 33, sedangkan resetCounter() dirancang mengembalikan counter dan progress ke nol.

5. Dokumentasi hasil memperlihatkan counter 5 untuk langkah kerja dan counter 13 untuk tugas. Pembahasan setState() dan GetX menunjukkan perbedaan penempatan state, pemicu pembaruan, serta pemisahan tanggung jawab kode.

PENUTUP

Praktikum Modul 11 memberikan pemahaman tentang hubungan antara data reaktif, controller, dan antarmuka Flutter. Melalui aplikasi tasbih digital, penerapan GetX dapat dipelajari pada kasus yang sederhana dan terarah. Pemisahan logika perhitungan dari tampilan menjadi dasar untuk mengembangkan aplikasi dengan pengelolaan state yang lebih terstruktur, dengan tetap memilih pendekatan yang sesuai dengan kebutuhan aplikasi.

LAMPIRAN VID LIVECODE : https://youtu.be/HomNvFHp6wY